TypeScript : architecture

I. Organiser une application React typée

Arborescence du projet

L'application repose sur une architecture définie dans `src/`. Le projet "fil rouge" sera porté sous Next.js (App Router) dans un TP ultérieur. La structure ci-dessous s'y prête déjà, pour qu'aucun réagencement ne soit nécessaire au moment venu.

```
mon-projet/
  public/          # fichiers statiques servis tels quels (images, favicon)
  src/
    app/           # routage : un sous-dossier par segment d'URL, layout racine
    components/    # composants React réutilisables
    services/      # appels réseau, accès aux données
    types/         # interfaces et types partagés
  tsconfig.json
```

Sous Next.js, `app/` devient le dossier de routage de l'App Router. Un sous-dossier y définit un segment d'URL, `page.tsx` fournit le contenu de la route et `layout.tsx` son enveloppe partagée.

Alias de chemins

Les alias de chemins (`@/components` , `@/types` , etc.) évitent les remontées relatives fragiles (`../../../components/Card`) et se configurent dans `tsconfig.json` via `paths` . Next.js les respecte nativement, sans configuration supplémentaire.

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/components/*": ["src/components/*"],
      "@/services/*": ["src/services/*"],
      "@/types/*": ["src/types/*"],
    },
  },
}
```

```
// Avant : chemin relatif fragile, casse si le fichier change de dossier
import { Card } from "../../../components/Card";

// Après : chemin d'alias stable, quel que soit l'emplacement du fichier
import { Card } from "@/components/Card";
```

Doc officielle TypeScript (Module Resolution - paths) : https://www.typescriptlang.org/tsconfig/#paths

Deux questions distinctes

Une arborescence répond en réalité à deux questions indépendantes, souvent confondues.

1. **Où vivent les fichiers d'une fonctionnalité ?** Réponses possibles : groupés par rôle technique, groupés par feature, ou groupés selon une méthode formalisée comme Feature-Sliced Design.
2. **Comment se range le catalogue d'interface réutilisable ?** C'est la question à laquelle répond l'atomic design.

Les deux réponses se combinent. Un projet organisé par features garde presque toujours un catalogue d'UI partagé, et ce catalogue se prête bien à une hiérarchie atomique. L'erreur consiste à appliquer deux découpages concurrents au même endroit, pas à les faire coexister à deux niveaux différents.

Groupement par type de fichier

Un dossier par rôle technique, quelle que soit la fonctionnalité concernée.

```
src/
  components/
    LoginForm/
      LoginForm.tsx
      LoginForm.module.scss
    SignupForm/
      SignupForm.tsx
      SignupForm.module.scss
  store/
    useAuthStore.ts
  services/
    auth.ts
```

Chaque composant a son propre dossier, avec ses styles co-localisés. C'est une bonne pratique en soi, et l'approche reste la plus simple à expliquer à un nouvel arrivant.

Le coût du groupement par type de fichier

Le problème est la dispersion d'une même fonctionnalité. Modifier l'authentification oblige à ouvrir `components/`, `store/` et `services/` en même temps, et la supprimer devient une chasse aux fichiers orphelins dans chacun de ces dossiers.

Ce coût grandit avec le nombre de features. Une application de dix écrans qui partagent le même domaine métier s'en accommode très bien.

La FAQ React refuse d'ailleurs de trancher et recommande de ne pas passer plus de cinq minutes sur la question au démarrage d'un projet, quitte à réorganiser plus tard.

Doc React (File Structure FAQ, site legacy) : https://legacy.reactjs.org/docs/faq-structure.html

Groupement par feature

Tout ce qui appartient à une fonctionnalité vit au même endroit.

```
src/
  features/
    auth/
      LoginForm.tsx
      useAuth.ts
      api.ts
      types.ts
    article-editor/
      EditorToolbar.tsx
      useAutosave.ts
      types.ts
  components/       # catalogue d'UI partagé par plusieurs features
  services/         # client HTTP et accès Supabase partagés
```

Ajouter une fonctionnalité crée un dossier. La supprimer en supprime un. Ce qui sert à plusieurs features remonte dans `components/` ou `services/`, jamais l'inverse.

Attention au raccourci fréquent : cette organisation légère n'est pas Feature- Sliced Design, qui est une spécification bien plus contraignante.

Feature-Sliced Design : les couches

Feature-Sliced Design (FSD) est une architecture standardisée pour l'ensemble du projet, pas une convention de rangement des composants. Elle impose six couches ordonnées.

```
src/
  app/          # entrypoint, providers, styles globaux, routage
  pages/        # pages complètes
  widgets/      # blocs autonomes livrant un cas d'usage entier
  features/     # fonctionnalités produit réutilisées
  entities/     # entités métier : user, article, comment
  shared/       # code réutilisable détaché du métier
```

La règle d'import est le coeur de la méthode. Un module ne peut importer que depuis les couches strictement inférieures. Une `feature` peut donc utiliser une `entity`, l'inverse est interdit.

Attention sous Next.js : `app/` et `pages/` sont déjà des noms réservés par le framework pour le routage. FSD impose alors une adaptation, documentée par le projet, et ne s'installe pas tel quel.

Doc officielle FSD : https://feature-sliced.design/docs/get-started/overview

Feature-Sliced Design : slices et segments

Chaque couche se découpe en **slices** par domaine métier, et chaque slice en **segments** par nature technique.

```
src/entities/article/
  ui/          # composants d'affichage de l'entité
  api/         # requêtes liées à l'entité
  model/       # état, types, logique métier
  lib/         # utilitaires internes
  index.ts     # public API de la slice
```

Deux slices d'une même couche ne peuvent pas s'importer l'une l'autre. Toute consommation externe passe par le `index.ts` de la slice, qui réexporte uniquement ce que la slice accepte d'exposer. La doc officielle déconseille explicitement les réexports génériques `export * from "./ui"`, qui rendent le contrat illisible.

Un linter officiel, `steiger`, vérifie ces règles.

Doc officielle FSD (Public API) : https://feature-sliced.design/docs/reference/public-api

Atomic design : la méthode

Atomic design est une méthodologie de conception d'interface formulée par Brad Frost. Elle fournit un vocabulaire partagé entre designers et développeurs, et reste indépendante de toute technologie. Ce n'est pas une convention de dossiers.

- **Atoms** : éléments HTML indivisibles. Un label, un input, un bouton.
- **Molecules** : petits groupes d'atoms formant une unité fonctionnelle, comme un champ de recherche réunissant label, input et bouton.
- **Organisms** : sections d'interface distinctes, assemblées de molecules et d'atoms. Un en-tête de site, par exemple.
- **Templates** : mise en page au niveau de la page, sans contenu réel. Un template fixe la structure et les contraintes de contenu, longueur de texte ou dimensions d'image.
- **Pages** : un template rempli de contenu représentatif. Leur rôle est de vérifier que le système tient avec du vrai contenu et ses cas limites.

Doc officielle : https://atomicdesign.bradfrost.com/chapter-2/

Atomic design transposé en dossiers

La transposition courante en arborescence est une convention d'équipe, pas une prescription de la méthode.

```
src/components/
  atoms/          # Button, Input, Icon
  molecules/      # SearchBar, FormField
  organisms/      # SiteHeader, ArticleCard
```

```tsx
// molecules/SearchBar.tsx compose deux atoms sans connaître leur usage final
function SearchBar({ onSearch }: { onSearch: (query: string) => void }) {
  const [value, setValue] = useState("");
  return (
    <div>
      <input value={value} onChange={(e) => setValue(e.target.value)} />
      <Button onClick={() => onSearch(value)}>Rechercher</Button>
    </div>
  );
}
```

Limite à connaître : la frontière entre molecule et organism est discutable au cas par cas, et les discussions de classement peuvent coûter plus cher que le rangement lui-même.

Choisir

Aucune approche n'est supérieure dans l'absolu.

SITUATION	CHOIX RAISONNABLE
Projet court, un seul domaine métier	Groupement par type de fichier
Périmètre mouvant, features ajoutées souvent	Groupement par feature
Grande équipe, besoin d'une règle vérifiable	FSD, avec son linter
Design system mature partagé avec les designers	Atomic design sur le catalogue d'UI

Les deux dernières lignes ne s'excluent pas. Sur un projet FSD, le catalogue atomique se place dans `shared/ui`.

Doc Patterns.dev (Design Patterns) : https://www.patterns.dev/